

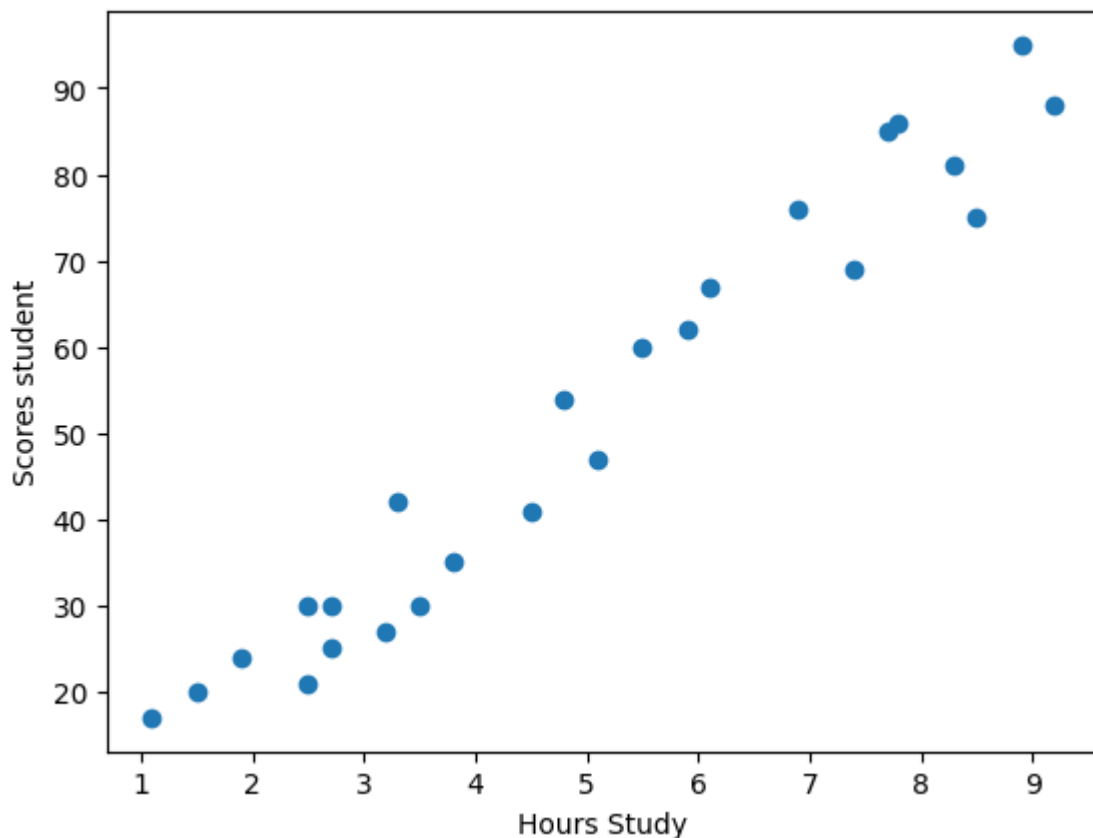
Build a Score Prediction Model Using Linear Regression

```
In [1]: # =====  
# IMPORT DATA FRAME  
# =====  
import pandas as pd  
import numpy as np  
  
data=pd.read_csv('student_scores.csv')  
data.head()
```

```
Out[1]:
```

	Hours	Scores
0	2.5	21
1	5.1	47
2	3.2	27
3	8.5	75
4	3.5	30

```
In [2]: # =====  
# visualisation  
# =====  
import matplotlib.pyplot as plt  
x =data["Hours"]  
y =data ["Scores"]  
  
plt.scatter(x,y)  
plt.xlabel("Hours Study")  
plt.ylabel("Scores student")  
plt.show()
```



The model shows a linear pattern.

```
In [3]: from sklearn.model_selection import train_test_split
from scratch.linear_algebra import Vector, dot
import tqdm
from scratch.linear_algebra import vector_mean
from scratch.gradient_descent import gradient_step
from scratch.statistics import de_mean
# predict value
def predict (x:Vector,beta:Vector):
    assert len(x)==len(beta)
    return dot (x,beta)

def error (x:Vector,beta:Vector,y:float):
    return predict(x,beta)-y

def error_gradient (x:Vector,y:float,beta:Vector):
    err =error(x,beta,y)
    return [2*err*x_i for x_i in x]

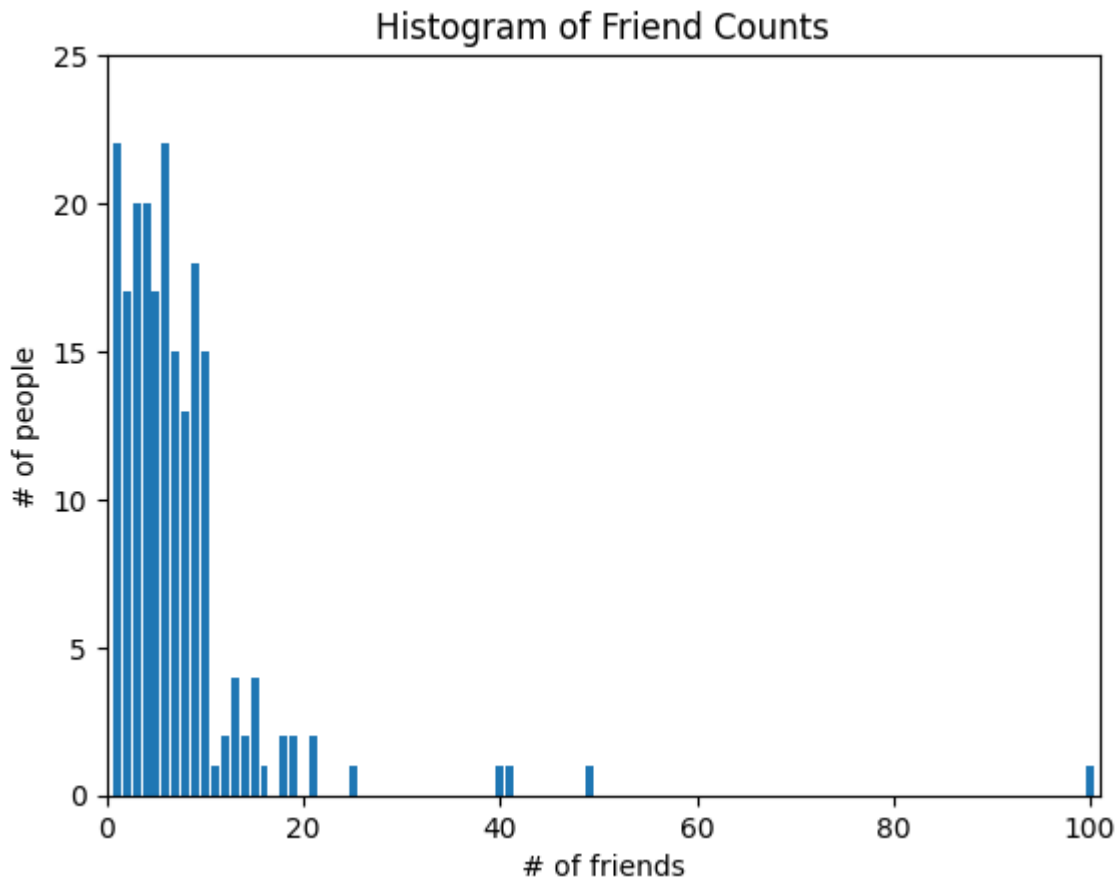
# Mean Absolute Error
def mae(xs:list[Vector],ys:list[Vector],beta:Vector):
    return sum(abs(error(x,beta,y)) for x,y in zip (xs,ys))/len(ys)

def MSE (xs:list[Vector],ys:list[Vector],beta:Vector):
    sq_error=sum(error (x,beta,y)**2 for x,y in zip(xs,ys))
    return sq_error/len(ys)

def RMSE( xs:list[Vector],ys:list[float],beta:Vector):
    return (MSE (xs,ys,beta))**0.5

def r_squared(xs:list[Vector],ys:list[float],beta:Vector):
    sum_of_error=sum(error(x,beta,y)**2 for x,y in zip (xs,ys))
```

```
total_sum_of_square=sum(v**2 for v in de_mean(ys))
return 1-(sum_of_error/total_sum_of_square)
```



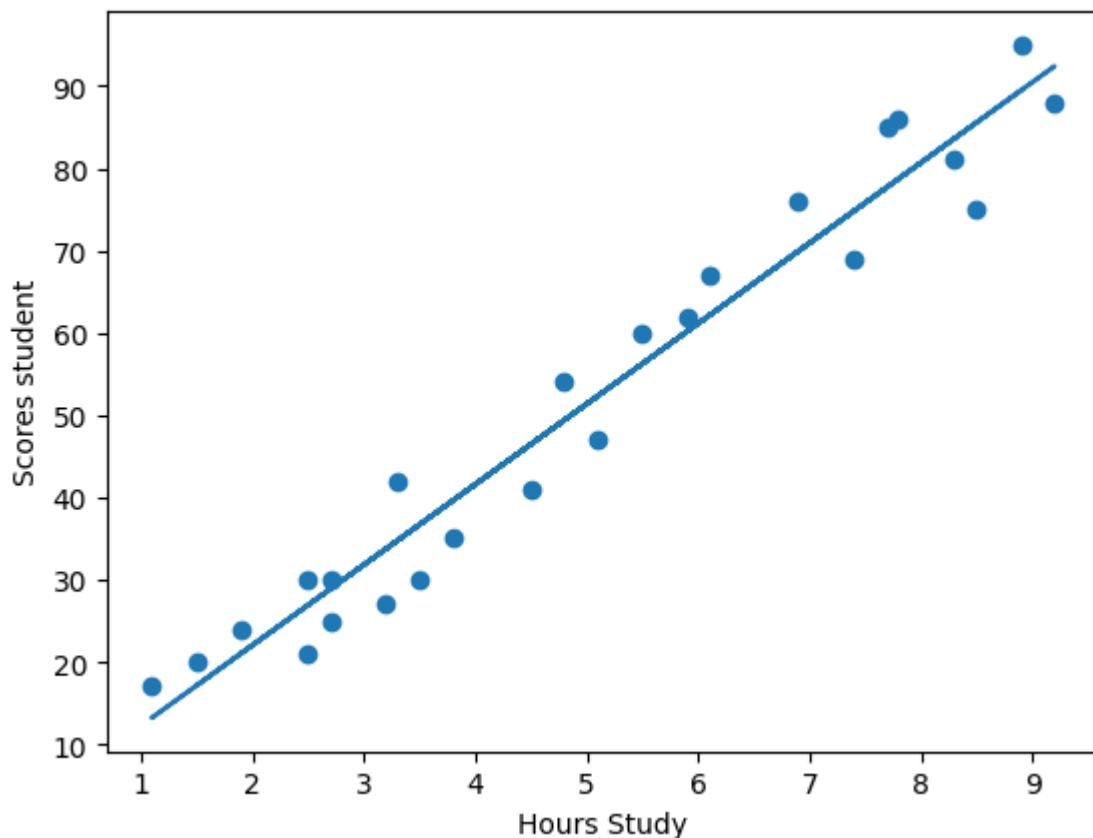
```
In [4]: # =====
# Model 1
# =====
from scratch.statistics import correlation, standard_deviation, mean
from scratch.linear_algebra import Vector

def least_squares_fit(x:Vector, y:Vector):
    beta = correlation(x,y) * standard_deviation(y)/standard_deviation(x)
    alpha=mean(y)-beta*mean(x)
    return alpha, beta

print (least_squares_fit(x,y))
alpha,beta=least_squares_fit(x,y)

xs=[[1,i] for i in x]
y_1=[x_i*beta+alpha for x_i in x]
plt.scatter(x,y)
plt.plot(x,y_1)
plt.xlabel("Hours Study")
plt.ylabel("Scores student")
plt.show()
beta=[alpha,beta]
```

(2.4836734053731817, 9.775803390787473)



```
In [5]: # =====
# value
# =====
print ('Result Model')
print (f'mae:{mae(xs,y,beta)}')
print (f'MSE :{MSE(xs,y,beta)}')
print (f"RMSE :{RMSE(xs,y,beta)}")
print (f"SCORE :{r_squared(xs,y,beta)}")
```

```
Result Model
mae:4.972805319866376
MSE :28.882730509245462
RMSE :5.374265578592619
SCORE :0.9529481969048356
```

```
In [6]: # =====
# Build Model 2
# =====
import random
import tqdm
from scratch.gradient_descent import gradient_step
from scratch.linear_algebra import Vector
guess=[random.random(),random.random()]
learning_rate = 0.001
num_epoch=1000

def predict (alpha:float, beta: float, x_i: float ):
    return beta*x_i + alpha
def error(alpha: float, beta :float, x_i:float, y_i: float):
    return predict (alpha,beta, x_i)- y_i
def sum_of_sqerror (alpha:float,beta:float, x: Vector, y : Vector):
    return sum(error(alpha, beta,x_i,y_i)**2 for x_i,y_i in zip (x,y))

with tqdm.trange(num_epoch)as t:
```

```

losss; 724.397993:  2%||          | 17/1000 [00:00<00:05, 169.62it/s]
losss; 722.068263: 100%|██████████| 1000/1000 [00:06<00:00, 151.08it/s]

```

```
nilai alpha : 2.483617629718142
nilai beta  : 9.775812393867412
```

A scatter plot illustrating a positive linear relationship between two variables. The x-axis is labeled from 1 to 9, and the y-axis is labeled from 10 to 90. A blue line of best fit is drawn through the data points, which are represented by blue dots. The data points show a clear upward trend, with some scatter around the line of best fit.

X	Y
1.1	17
1.5	20
1.9	24
2.5	21
2.5	30
2.7	25
2.7	30
3.2	27
3.3	42
3.6	30
3.8	35
4.5	41
4.8	54
5.1	47
5.5	60
5.9	62
6.1	67
6.9	76
7.4	69
7.6	85
7.8	86
8.3	81
8.5	75
8.9	95
9.1	88

Result Model

```
-----  
-  
TypeError                                Traceback (most recent call las  
t)  
Cell In[9], line 2  
      1 print ('Result Model')  
----> 2 print (f'mae:{mae(xs,y,beta)}')  
      3 print (f'MSE :{MSE(xs,y,beta)}')  
      4 print (f"RMSE :{RMSE(xs,y,beta)}")  
  
Cell In[3], line 21, in mae(xs, ys, beta)  
     20 def mae(xs:list[Vector],ys:list[Vector],beta:Vector):  
--> 21     return sum(abs(error(x,beta,y))for x,y in zip (xs,ys))/len(ys)  
  
Cell In[3], line 21, in <genexpr>(.  
     20 def mae(xs:list[Vector],ys:list[Vector],beta:Vector):  
--> 21     return sum(abs(error(x,beta,y))for x,y in zip (xs,ys))/len(ys)  
  
TypeError: error() missing 1 required positional argument: 'y_i'
```